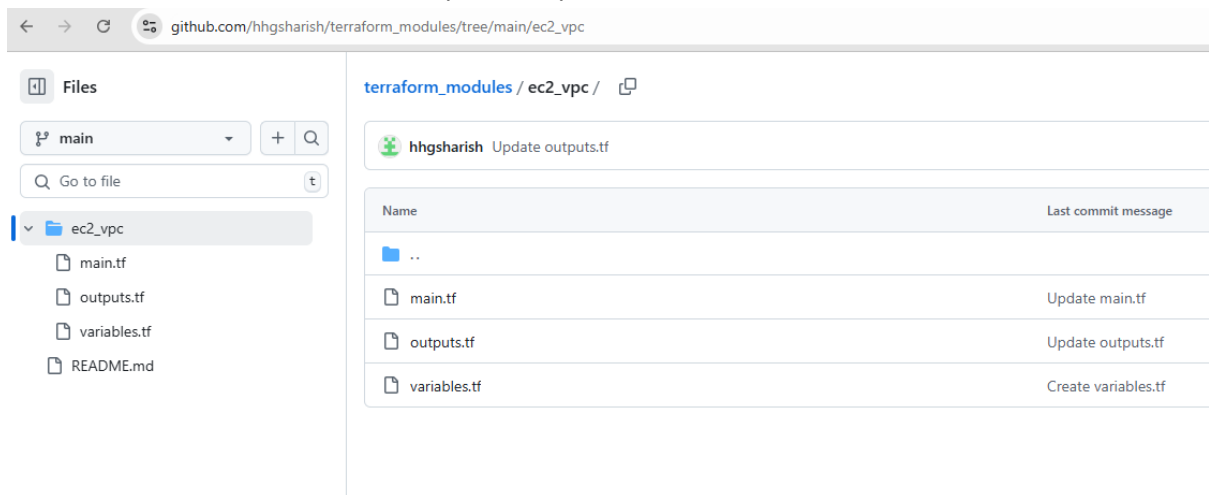


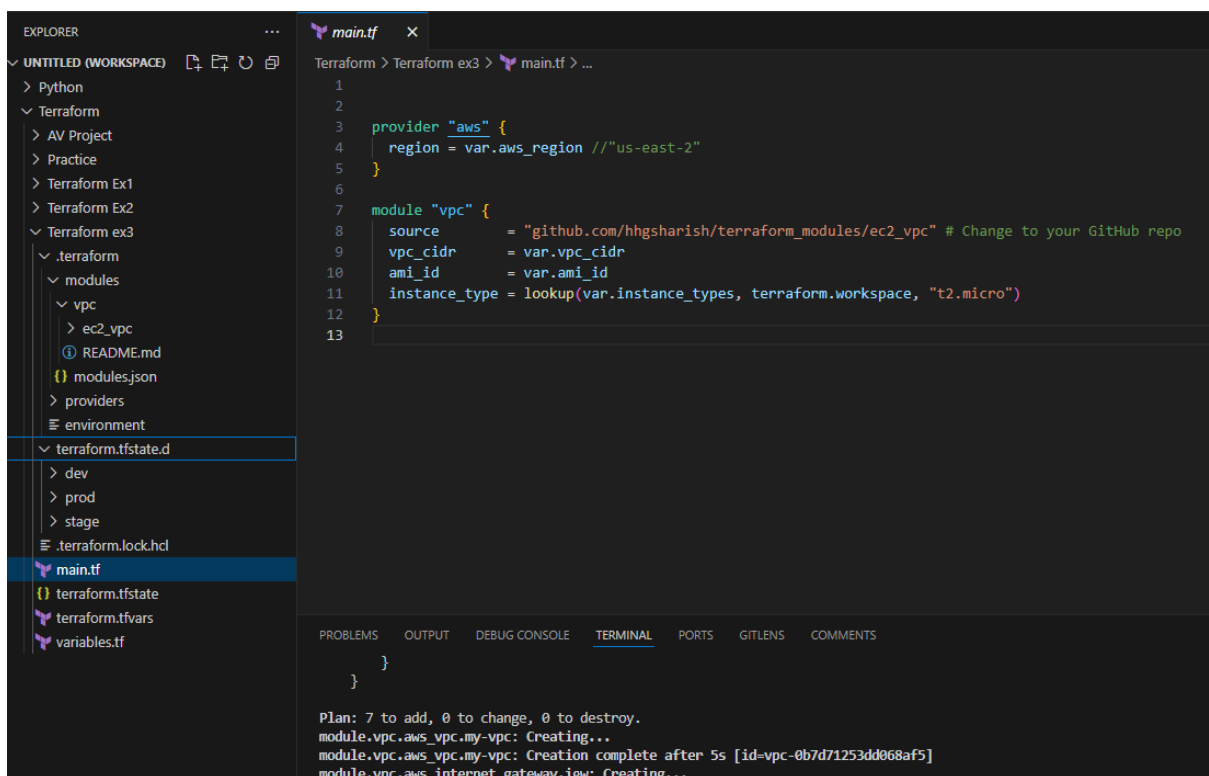
Modules & workspace assignment

- Create a Terraform module that provisions a VPC, security groups (allowing SSH & HTTP) , and an EC2 instance . Upload the module code to your github repo and reference the github source for your module block
- The setup should use Terraform workspaces for different environments (dev, stage, prod) and allow users to specify the CIDR block of their choice.
- Make use of -
- Variables for user-defined inputs.
- Locals for computed values.
- Functions to format values dynamically.



The screenshot shows a GitHub repository page for the path `terraform_modules / ec2_vpc /`. The left sidebar displays the file structure with files `main.tf`, `outputs.tf`, `variables.tf`, and `README.md`. The main content area shows a commit history table for the repository.

Name	Last commit message
..	
main.tf	Update main.tf
outputs.tf	Update outputs.tf
variables.tf	Create variables.tf



The screenshot shows a VS Code editor with a Terraform module `main.tf` open. The left sidebar shows the Explorer view with the file structure. The main editor shows the code for `main.tf`, which defines a VPC module and an EC2 instance. The bottom panel shows the output of the Terraform command, indicating that the VPC and EC2 instance were created successfully.

```
1
2
3 provider "aws" {
4   region = var.aws_region //"us-east-2"
5 }
6
7 module "vpc" {
8   source      = "github.com/hhgsharish/terraform_modules/ec2_vpc" # Change to your GitHub repo
9   vpc_cidr    = var.vpc_cidr
10  ami_id      = var.ami_id
11  instance_type = lookup(var.instance_types, terraform.workspace, "t2.micro")
12 }
13
```

Plan: 7 to add, 0 to change, 0 to destroy.
module.vpc.aws_vpc.my-vpc: Creating...
module.vpc.aws_vpc.my-vpc: Creation complete after 5s [id=vpc-0b7d71253dd068af5]
module.vpc.aws_internet_gateway.igw: Creating...

terraform.tfvars X

Terraform > Terraform ex3 > terraform.tfvars > ami_id

```
1 aws_region = "us-east-2"
2 vpc_cidr   = "10.0.0.0/16"
3 ami_id     = "ami-0cb91c7de36eed2cb"
```

variables.tf X

Terraform > Terraform ex3 > variables.tf > variable "ami_id" > type

```
1 variable "aws_region" {
2   description = "AWS region"
3   type       = string
4 }
5
6 variable "vpc_cidr" {
7   description = "CIDR block for the VPC"
8   type       = string
9 }
10
11 variable "ami_id" {
12   description = "AMI ID for EC2"
13   type       = string
14 }
15
16 variable "instance_types" {
17   description = "Instance type based on the workspace"
18   type       = map(string)
19
20   default = {
21     dev  = "t2.micro"
22     stage = "t3.small"
23     prod = "t3.medium"
24   }
25 }
26
```

```
PS C:\devops\Terraform\Terraform ex3> terraform workspace new dev
Created and switched to workspace "dev"!
```

You're now on a new, empty workspace. Workspaces isolate their state, so if you run "terraform plan" Terraform will not see any existing state for this configuration.

```
PS C:\devops\Terraform\Terraform ex3> terraform init
Initializing the backend...
Initializing modules...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.89.0
```

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
PS C:\devops\Terraform\Terraform ex3> terraform apply -auto-approve
```

Terraform used the selected providers to generate the following execution plan. Resource
+ create

for this configuration.

```
PS C:\devops\Terraform\Terraform ex3> terraform workspace select prod
```

```
PS C:\devops\Terraform\Terraform ex3> terraform workspace show
```

prod

```
PS C:\devops\Terraform\Terraform ex3> terraform init
```

Initializing the backend...

Initializing modules...

Initializing provider plugins...

- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v5.89.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
PS C:\devops\Terraform\Terraform ex3> terraform apply -auto-approve
```

Terraform used the selected providers to generate the following execution plan. Resource
+ create

Instances (3) [Info](#)

All states ▾

☐	Name 	Instance ID	Instance state ▾	Instance type ▾	Status check
☐	prod-instance	i-06f94adb24c5f0497	 Running  	t3.medium	 3/3 checks passed
☐	default-instance	i-056dbcf7a28226960	 Running  	t2.micro	 2/2 checks passed
☐	dev-instance	i-0c79057527d76380a	 Running  	t2.micro	 Initializing